
PHASEGATE: Phase-Aware CPU Retrieval Scheduling for On-Device LLMs on Unified Memory

Seoyoon Yum

KAIST

`gilbertyum@kaist.ac.kr`

Sehoon Kim

KAIST

`sehoonkim@kaist.ac.kr`

Abstract

On-device assistants run GPU-based LLM inference alongside CPU retrieval on unified-memory systems. Under a saturated local-retrieval workload, four concurrent retrieval workers raise 95th-percentile (p95) decode latency by 60–61% on two M4 systems, whereas prefill latency rises by only 5.7–6.9%. We study LLM phase as an admission signal for independent CPU retrieval under controlled LLM workloads. PHASEGATE calibrates separate concurrency limits for prefill and decode, selecting four and one on our base-M4 configuration. Under a backlogged queue, it achieves $2.0\times$ the aggregate retrieval throughput of the best tested feasible fixed policy, with both p95 LLM latency metrics within $1.25\times$ their no-retrieval baselines in all seven held-out runs. A phase-blind control, TimeGate, uses the same two limits on a calibration-derived schedule without observing LLM phase. It achieves similar retrieval throughput but violates the output-token latency limit in every run. M2 and M2 Pro Mac minis reproduce the policy ordering, while output-length sweeps show that the advantage narrows as decode occupies more of each request.

1 Introduction

Compact models such as Apple’s foundation model for Apple Intelligence and Google’s Gemini Nano support on-device inference [1, 2]. An on-device assistant also uses the CPU for retrieval, contextual-memory lookup, indexing, parsing, and tool execution. Representative agentic applications spend substantial time in CPU-side tools [3], and on-device vector-search systems must coordinate interactive queries with background ingestion and index maintenance [4].

Server accelerators stream model state from GPU-local high-bandwidth memory (HBM), separate from host DRAM used by CPU tools [5, 6]. Apple Silicon instead exposes CPU and GPU to unified memory [7], making CPU pointer chasing and GPU weight streaming compete for memory service. LLM inference has two phases: prefill processes prompt tokens in parallel and reuses weight loads, while decode emits one token at a time and repeatedly streams model state.

BWLOCK++ regulates CPU memory traffic during GPU execution using profiled bandwidth budgets [8]. FusionML splits prefill computation across the Apple-Silicon CPU and GPU while keeping decode on the GPU [9]. We study a different control target: phase-dependent admission of independent CPU retrieval while keeping LLM execution on the GPU.

We use FAISS with a Hierarchical Navigable Small World (HNSW) index for approximate nearest-neighbor search [10, 11]. HNSW performs irregular graph traversal; our saturated queue measures capacity for retrieval independent of the concurrent generation, with 5%, 25%, and 100% retrieval-active time. On a fanless M4 MacBook Air, four HNSW workers increase p95 time-to-first-token (TTFT), our prefill-sensitive metric, by 5.7%, but time-per-output-token (TPOT), our decode metric, by 61%; a fan-cooled base-M4 Mini preserves the direction.

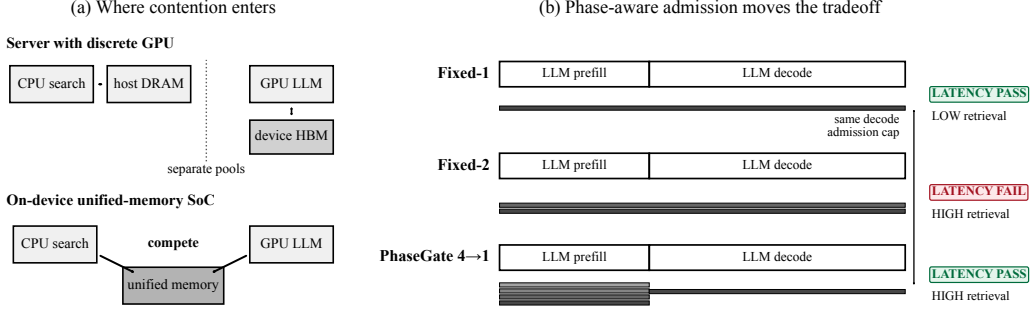


Figure 1: (a) A discrete-GPU server’s CPU search and GPU LLM use separate memory pools, whereas Apple Silicon shares unified memory. (b) On the measured base-M4 configuration, explicit PASS/FAIL labels show the tradeoff: Fixed-1 meets both latency limits but restricts retrieval, while Fixed-2 retrieves more but violates the output-token limit. PHASEGATE raises its admission cap during prefill and lowers it during decode; in-flight work may cross the boundary. Schematic; PASS denotes measured compliance under the evaluated conditions.

A conventional retrieval pool uses one maximum concurrency throughout an LLM request. We write Fixed- n for a phase-oblivious concurrency cap of n , and $p \rightarrow d$ for PHASEGATE’s prefill and decode admission caps. On the primary M4, Fixed-1 meets both latency limits, while the faster Fixed-2 violates output-token latency. PHASEGATE calibrates separate caps (Figure 1), selecting 4→1 on this configuration. This pair is deployment-specific, and we compare it with a phase-blind timer that alternates the same limits without observing LLM phase.

Contributions. (1) We quantify prefill–decode interference asymmetry using HNSW on two M4 systems and add a decode-only access-pattern control on the M4 Air. (2) We evaluate a simple phase-based admission policy and isolate the value of phase alignment with a time-matched control. (3) We evaluate the scheduler on M4, M2, and M2 Pro Minis, vary output length, and synthetically sweep the fraction of time with retrieval demand.¹

2 Observation: Phase-Asymmetric Interference

Experimental Setup. We run 4-bit Qwen2.5-1.5B using MLX-LM v0.31.3 [12, 13] alongside a read-only FAISS HNSW index of 100,000 384-dimensional vectors with graph connectivity $M = 32$. On the M4 Mini, controlled synthetic traces use 2,048 input token IDs followed by 128 sequential single-token decode forward passes. Tokenization, sampling, and natural-language output quality are outside this workload; Appendix A.1 details the execution loop. Retrieval uses two or four concurrent single-threaded FAISS HNSW calls, each processing 16 queries. Concurrency caps count calls; throughput in queries per second (QPS) counts individual vector queries.

The queue remains nonempty during active periods, measuring aggregate retrieval capacity under an LLM-latency constraint. Retrieval is independent of the concurrent generation, not prerequisite retrieval for that response. Lower admission during decode can delay individual searches, especially for long outputs; we do not measure retrieval response time or end-to-end RAG latency.

¹Code, experiment scripts, and processed results: <https://github.com/SeoyoonYum/PhaseGate>.

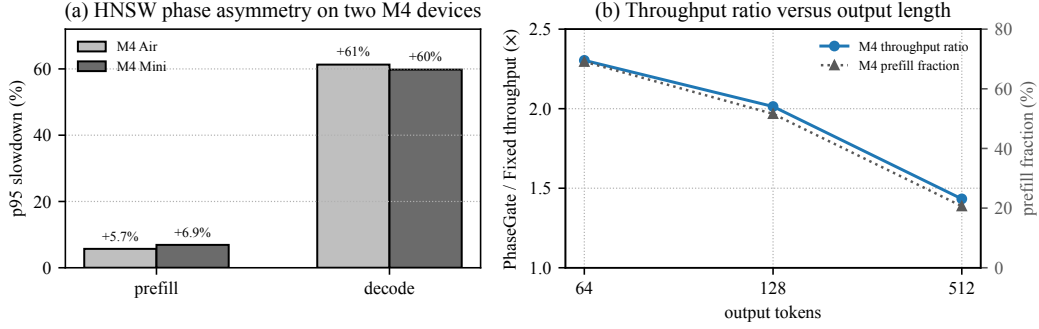


Figure 2: (a) Four concurrent HNSW workers show the same p95 asymmetry on a fanless M4 Air and fan-cooled base-M4 Mini; each Mini bar is the median of three runs with 300 LLM requests per run. (b) On the base-M4 Mini, PHASEGATE/Fixed throughput falls with output length (five pairs per length); the right axis shows the M4 prefill fraction.

Decode Is More Sensitive to Concurrent Retrieval. On the M4 Mini, we first measure LLM latency without retrieval, then co-run either two or four concurrent HNSW calls. Each concurrency setting uses three runs of 300 LLM requests, with the two settings evaluated in randomized order. TTFT spans request start to the first completed decode step; TPOT summarizes subsequent step-to-step gaps. With two concurrent calls on the M4 Mini, p95 TTFT increases by only 1.3%, whereas p95 TPOT increases by 31%. With four, Figure 2a shows 5.7% versus 61% on the Air and 6.9% versus 60% on the Mini; the Mini shows greater decode sensitivity in every repetition. This pattern is consistent with prefill amortizing weight reads while decode has little reuse.

Access Pattern Matters Too. After establishing the phase asymmetry, we isolate access pattern by fixing the hardware to the M4 Air and examining decode only. An address-jumping random gather reports just 2.4 GB/s of logical bandwidth but increases p95 decode latency by 29%, whereas a contiguous 51 GB/s scan increases it by 27%. Thus byte rate alone is a poor interference proxy: irregular access can cause comparable slowdown to a much higher-rate sequential scan, which is relevant to graph-based retrieval such as HNSW [11].

3 PHASEGATE

PHASEGATE uses the observed LLM phase to admit queued retrieval under separate prefill and decode caps.

Policy and Calibration. Let p and d be the prefill and decode concurrency limits. A phase-oblivious scheduler uses one limit throughout (e.g., $p = d = 1$); PHASEGATE permits $p > d$. The deployment supplies a baseline-relative budget B ; $B = 1.25$, for example, allows p95 TPOT and TTFT to rise by at most 25% over their no-retrieval baselines. Calibration tests candidate (p, d) pairs and selects the pair with the highest retrieval QPS among those that consistently meet both limits. Fixed caps assume conditions comparable to calibration, including background activity and memory pressure. Phase labels do not measure current contention: compliance under the evaluated conditions is not a guarantee under changing load. Material changes to hardware, model, sequence lengths, retrieval, or load require recalibration.

Runtime. Our instrumented MLX-LM harness emits prefill and decode events; PHASEGATE admits a queued CPU workload (e.g., a search) only while the active count is below that phase’s limit. Already-running CPU workloads are not preempted: when the phase limit decreases, they finish while new work waits until the active count falls below the new limit. The decode cap is thus an admission threshold, not an instantaneous bound on active work. Effective isolation requires residual work to drain quickly relative to decode; jobs comparable to prefill duration are more likely to cross the boundary, and long or variable jobs can interfere well into decode. We request no core affinity or thread-priority (QoS) override.

Control Target and Evaluation. PHASEGATE controls the number of admitted retrieval calls, while LLM execution remains on the GPU. We measure how phase-dependent admission changes retrieval throughput under LLM latency constraints. Fixed caps provide a constant-concurrency comparison, while TimeGate uses the same two caps and a calibration-matched schedule to isolate the effect of phase alignment within this admission design. Appendix B discusses the broader literature.

4 Experiments

Experimental Setup. We use the same base-M4 Mini, model, HNSW index, and synthetic workload as in Section 2. We compare fixed caps, PHASEGATE, and TimeGate under a backlogged retrieval queue. For M4, we first compute the p95 inter-token gap within each request, then take the p95 of those values across requests. Run-level p95 TTFT is computed directly across requests. Appendix A gives setup details and metric definitions.

4.1 Main Experiments

We calibrate FIXED-0, FIXED-1, FIXED-2, FIXED-4, and PHASEGATE 2→0, PHASEGATE 2→1, PHASEGATE 4→0, PHASEGATE 4→1, and PHASEGATE 4→2 in three randomized 100-request runs per candidate. A candidate is eligible only if both p95 latency metrics remain at or below the baseline-relative budget B in every run. Among tested budgets 10–45% above baseline, $B = 1.25$ is the smallest at which both a fixed policy with nonzero retrieval throughput and a PHASEGATE policy are eligible. We use it as a controlled comparison point, not a universal latency threshold.

Table 1: Base-M4 calibration sweep at $B = 1.25$. Throughput is the median of three runs; TPOT is the maximum per-run p95 ratio. Unrounded ratios determine eligibility; † exceeds B . All maximum TTFT ratios are < 1.1 ; bold policies are selected.

Fixed cap	CPU throughput (kQPS) ↑	Worst TPOT ↓ (× baseline)	PHASEGATE caps	CPU throughput (kQPS) ↑	Worst TPOT ↓ (× baseline)
0	0	1.021	2→0	0.79	1.265†
1	0.78	1.247	2→1	1.1	1.225
2	1.4	1.393†	4→0	1.2	1.271†
4	2.1	1.690†	4→1	1.6	1.177
			4→2	1.8	1.384†

Table 1 exposes the central tradeoff. Raising a constant cap from one to two or four increases CPU throughput from 0.78 to 1.4 or 2.1 kQPS, but worst TPOT rises to $1.393\times$ or $1.690\times$ baseline, exceeding B . PHASEGATE 4→1 reaches 1.6 kQPS at $1.177\times$ worst TPOT; raising its decode cap to two reaches 1.8 kQPS but violates B at $1.384\times$. Calibration therefore selects FIXED-1 and PHASEGATE 4→1, the highest-throughput eligible policy in each family. The zero-decode-cap policies have higher TPOT than their one-cap counterparts in all three runs. The cause is unresolved; calibration evaluates each candidate directly without assuming monotonicity (Appendix A.2).

We hold those choices fixed and evaluate them in seven paired randomized comparisons with 250 LLM requests per policy. FIXED-1 and PHASEGATE 4→1 meet both latency limits in all seven runs, but PHASEGATE 4→1 reaches 1.6 kQPS versus 0.78 kQPS by increasing concurrency only during prefill (Table 2).

4.2 Additional Analysis

Phase-Blind Control. Higher average concurrency alone could explain the throughput increase, so we designed a phase-blind control called TimeGate. TimeGate replays a wall-clock schedule with the same high and low caps. Its high-cap time fraction and switching rate match PHASEGATE’s calibration schedule; it does not observe runtime LLM phase. TimeGate reaches a similar 1.5 kQPS, yet raises p95 TPOT by 60% and meets both latency limits in 0/7 runs. Its high-cap state overlaps 23% of decode, compared with 0.003% for PHASEGATE; these are admission-state durations, not bounds on active CPU work. This control supports the value of phase alignment in the tested design.

Table 2: Base-M4 evaluation at $B = 1.25$. Latency, throughput, and transition rows report medians across seven paired randomized repetitions with 250 requests per policy; “7/7” means both latency limits pass in all seven.

Metric	FIXED-1	PHASEGATE 4→1	TimeGate 4/1
Runs meeting both limits	7/7	7/7	0/7
p95 TPOT increase (%)	17	18	60
p95 TTFT increase (%)	0.61	4.6	4.3
CPU throughput (kQPS)	0.78	1.6	1.5
p95 transition gap (ms)	18	23	24

Intermittent Retrieval Demand. We make retrieval active for 5%, 25%, or 100% of wall-clock time, retaining a backlog during each active burst (five matched three-policy comparisons per duty; Appendix A.4). The median full-wall-clock PHASEGATE/Fixed throughput ratio remains $2.0\times$ at all three duties. FIXED-1 and PHASEGATE each meet both limits in all 15 runs, while TimeGate passes none; at 5% duty, idle time lowers the absolute throughput to 79 versus 40 QPS without erasing the ratio.

Output-Length Sensitivity. We keep each device’s selected caps fixed while varying output length to change the balance between prefill and decode. On M4, PHASEGATE/Fixed throughput falls from $2.3\times$ at 64 tokens to $2.0\times$ at 128 and $1.4\times$ at 512 as prefill’s share of LLM processing time falls from 69% to 52% and 21% (Figure 2b). The M2 Mini shows the same decreasing direction, from $1.8\times$ at 128 tokens to $1.4\times$ at 512 (see Appendix A.3); devices are analyzed separately and the association is descriptive.

Cross-Device Replication. Table 3 summarizes the device-specific comparisons. M2 and M2 Pro summarize each request’s token gaps by their mean, while M4 uses their p95. The run-level p95 therefore tests different latency criteria on these devices (Appendix A.1). The M2 Pro comparison uses a tighter budget and yields $2.5\times$, but lacks TimeGate and pageout-free pairs, so it supports only the policy ordering.

Table 3: Mac mini evaluations. Throughput entries are median paired run ratios of PHASEGATE to each control (two significant digits); passing runs meet both device-specific latency criteria (Appendix A.1), and “–” denotes an unevaluated control.

Device	B	vs. Fixed throughput ($\times$)	vs. TimeGate throughput ($\times$)	Fixed passing runs	PHASEGATE passing runs	TimeGate passing runs
Base-M4 Mini	1.25	2.0	1.0	7/7	7/7	0/7
Base-M2 Mini	1.25	1.8	1.0	5/5	5/5	0/5
M2 Pro Mini	1.10	2.5	–	7/7	7/7	–

4.3 Non-Preemptive Work and Transition Costs

Retrieval That Continues into Decode. When PHASEGATE switches from the prefill cap of four to the decode cap of one, calls already admitted continue. We examine this remaining work in the seven held-out PHASEGATE 4→1 traces, counting a call as active from admission to completion.

Drain time measures the delay from decode entry until the active count falls to at most one and stays there for the rest of decode. It is zero if the count never exceeds one during decode. Table 4 also reports the service time of each 16-query FAISS call, excluding queue waiting, and the delay until all calls active at decode entry finish. Appendix A.5 gives the trace-accounting details.

Table 4: CPU service and drain times from seven held-out PHASEGATE 4→1 traces (250 requests per run). The p95 column reports the median of seven per-run p95s; maxima span all seven traces.

Measure	Median run p95 (ms)	Observed maximum (ms)
16-query chunk service time	27.7	34.3
Drain time to at most one active call	26.6	31.5
Completion of all calls active at decode entry	29.0	32.2

The median per-run p95 drain time is 26.6 ms; the longest observed drain is 31.5 ms. These delays are short relative to median phase durations of 1857 ms for prefill and 1739 ms for decode (medians of per-run medians).

Active calls exceed one during 1.06% of decode time, although the requested cap is four during only 0.003% (both medians across runs). This difference reflects calls that continue after the admission cap falls. The observed maxima characterize these short HNSW calls and do not bound longer or more variable workloads. Such workloads, or shorter phases, require new calibration and service-time-tail measurements.

GPU Transition and Memory Costs. An in-memory monotonic clock records token and phase timestamps without a monitoring process, allowing us to measure the gap from prefill completion to the first decoded token and two tail token-gap metrics. The median paired increase in per-run p95 transition gap over Fixed-1 is 5.2 ms (0.29% of baseline p95 TTFT). The p99 inter-token gap and p95 per-request maximum token gap rise by less than 1% (median paired changes). CPU drain times describe residual admitted work; GPU token gaps describe delays in model execution. No swap occurs and memory pressure remains normal; pageouts remain a caveat, but removing any matched comparison still yields $2.0\times$.

Validity Checks. Fresh no-retrieval baselines vary by about 1.0% within the continuous and duty-sweep M4 studies. Swap growth, abnormal pressure, thermal warnings, corrupt traces, incorrect policy behavior, or severe drift invalidate a run, with one retry; small system-wide pageouts remain warnings. All planned policy runs yield valid measurements, and invalid attempts remain in the artifact.

5 Limitations and Conclusion

Limitations. The full protocol uses one base-M4 Mini; narrower tests use one each of M2 Mini, M2 Pro Mini, M4 Air, and M1 Air. We evaluate one 1.5B 4-bit LLM with 2,048-token inputs; synthetic token traces do not test natural-language generation or sampling. Scheduler and HNSW tests use read-only HNSW and device-specific caps. The duty sweep uses backlogged bursts rather than random arrivals or full agent workflows. Protocol and pageout differences prevent us from isolating cooling effects or ruling out effects from paging. We do not evaluate long non-preemptive jobs, retrieval waiting-time tails, or online adjustment of the caps under changing background contention.

Conclusion. Across the three synthetic duties, PHASEGATE delivers $2.0\times$ the median aggregate retrieval throughput of Fixed-1 while meeting the measured LLM latency budgets. TimeGate’s similar throughput and greater decode slowdown support phase as a useful admission signal under these conditions.

References

- [1] Tom Gunter et al. Apple Intelligence foundation language models. *arXiv preprint arXiv:2407.21075*, 2024.
- [2] Gemini Team et al. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.
- [3] Ritik Raj, Souvik Kundu, Ishita Vohra, Hong Wang, and Tushar Krishna. Towards understanding, analyzing, and optimizing agentic AI execution: A CPU-centric perspective. *arXiv preprint arXiv:2511.00739*, 2025.
- [4] Xinkui Zhao, Qingyu Ma, Yifan Zhang, Hengxuan Lou, Sai Liu, Chang Liu, Guanjie Cheng, Naibo Wang, and Yueshen Xu. MUSE: A heterogeneity-aware multimedia search engine for mobile SoCs. In *Proceedings of the 34th ACM International Conference on Multimedia (MM '26)*, 2026. doi: 10.1145/3767308.3835897. arXiv:2511.19192.
- [5] NVIDIA. NVIDIA H100 Tensor Core GPU architecture. White paper, NVIDIA, 2022.
- [6] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 193–210, 2024.
- [7] Apple. Apple unleashes M1. Apple Newsroom, 2020. URL <https://www.apple.com/newsroom/2020/11/apple-unleashes-m1/>.
- [8] Waqar Ali and Heechul Yun. Protecting real-time GPU kernels on integrated CPU–GPU SoC platforms. In *30th Euromicro Conference on Real-Time Systems (ECRTS)*, volume 106 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 19:1–19:22, 2018. doi: 10.4230/LIPIcs.ECRTS.2018.19.
- [9] Om Mohite. FusionML: Prefill, not decode—mechanism and boundaries of CPU+GPU co-execution on unified-memory Apple Silicon. *arXiv preprint arXiv:2607.22785*, 2026.
- [10] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021. doi: 10.1109/TBDDATA.2019.2921572.
- [11] Yu A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. doi: 10.1109/TPAMI.2018.2889473.
- [12] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [13] MLX Community. MLX-LM: Run LLMs with MLX. Software, version 0.31.3, 2026. URL <https://github.com/ml-explore/mlx-lm/tree/v0.31.3>.
- [14] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *51st ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, pages 118–132, 2024. doi: 10.1109/ISCA59077.2024.00019.
- [15] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput–latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 117–134, 2024.
- [16] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, pages 19274–19286, 2023.
- [17] Tong Xin, Xinrui Shi, Mingkai Dong, and Zeyu Mi. Inference in the shadows: Taming memory bandwidth contention in mobile LLM inference with Sereno. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 2299–2318, 2026.
- [18] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. Microsecond-scale preemption for concurrent GPU-accelerated DNN inferences. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 539–558, 2022.

- [19] Apple. Managing the on-device foundation model’s context window. Apple Developer Documentation, Technical Note TN3193, 2026. URL <https://developer.apple.com/documentation/Technotes/tn3193-managing-the-on-device-foundation-model-s-context-window>.
- [20] Taehwan Park, Geonho Lee, and Min-Soo Kim. MobileRAG: A fast, memory-efficient, and energy-efficient method for on-device RAG. *arXiv preprint arXiv:2507.01079*, 2025.
- [21] Tianxi Lu and Sherief Reda. Agentic CPU–GPU scheduling for heterogeneous AI workloads. *arXiv preprint arXiv:2607.22242*, 2026.
- [22] Yifei Wang, Hancheng Ye, Yechen Xu, Cong Guo, Chiyue Wei, Qinsi Wang, Dongting Li, Tingjun Chen, Hai Li, Danyang Zhuo, and Yiran Chen. MARS: Efficient, adaptive co-scheduling for heterogeneous agentic systems. *arXiv preprint arXiv:2604.26963*, 2026.

A Experimental Details and Additional Results

A.1 Setup and Latency Metrics

Table A1: Fan-cooled scheduler devices and shared setup (P/E CPU cores; UMA unified memory).

M4 primary	Mac mini (Mac16,10); 4P+6E CPU, 10-core GPU, 16 GB UMA
M2 replication	Mac mini (Mac14,3); 4P+4E CPU, 10-core GPU, 16 GB UMA
M2 Pro earlier	Mac mini (Mac14,12); 6P+4E CPU, 16-core GPU, 16 GB UMA
M4 runtime	macOS 26.5/26.6.1 (continuous study/duty sweep); MLX 0.31.2; MLX-LM 0.31.3; FAISS 1.14.3; MLX memory limit 5.5 GB
M4 model	mlx-community/Qwen2.5-1.5B-Instruct-4bit, revision 8b403126fc14f14cfc99bb4cfa72ecbc129ea677
Workload	2,048 input tokens/128 decode steps; 100k 384-D vectors; queue kept nonempty

Synthetic execution workload. The M4 Mini harness constructs deterministic input token IDs and prefills the transformer body to build the KV cache, omitting the vocabulary output head during prefill. Each decode step evaluates the full model on a fixed single-token input, grows the KV cache, synchronizes execution, and records a timestamp. It does not sample logits or feed generated tokens back into the model. Here, output length counts these decode steps; “token timestamps” mark completed model evaluations. This controls execution shape without measuring natural-language generation quality.

Latency metrics. Let s_r be request r ’s start time and $t_{r,i}$ its i th decode-step timestamp, recorded after model execution and synchronization. TTFT is $t_{r,1} - s_r$; inter-token gaps are $t_{r,i} - t_{r,i-1}$ for $i \geq 2$. For the M4 Mini evaluations, run-level p95 TPOT is $Q_{.95,r}[Q_{.95,i}(t_{r,i} - t_{r,i-1})]$, where $Q_{.95}$ denotes the 95th percentile; p95 TTFT is $Q_{.95,r}(t_{r,1} - s_r)$. M4 thus takes the p95 within each request and then across requests; the first-token boundary gap is reported separately. The earlier M2 and M2 Pro protocols use the mean within each request and the p95 across requests. They measure TTFT from recorded request arrival to the first token. Each device uses a matching no-retrieval baseline. These comparisons establish policy ordering within each protocol; their tail-latency criteria differ.

A.2 Calibration and Non-monotonicity

Table A2 provides the three randomized 100-request runs per candidate underlying Table 1. All ratios use the same no-retrieval reference, the median of five baseline runs’ p95 values: TPOT 12.046 ms and TTFT 1799.140 ms (rounded here; normalization uses full precision). Throughput is the median across runs; feasibility requires both unrounded p95 ratios to be at most 1.25 in every run. The maximum TTFT ratios are below 1.1 for every candidate, so TPOT determines eligibility here.

Table A2: Per-repetition calibration results. Ratios are rounded to three decimals for display; eligibility uses unrounded values. Run indices are repetition identifiers, not a simultaneous paired comparison.

Policy	Median kQPS	p95 TPOT / baseline			Max TTFT / baseline	Eligible at $B = 1.25$
		Run 0	Run 1	Run 2		
FIXED-0	0.000	1.009	0.996	1.021	0.980	Yes
FIXED-1	0.784	1.213	1.235	1.247	1.008	Yes
FIXED-2	1.379	1.325	1.294	1.393	1.016	No
FIXED-4	2.136	1.667	1.690	1.667	1.074	No
PHASEGATE 2→0	0.793	1.261	1.265	1.258	1.002	No
PHASEGATE 2→1	1.146	1.218	1.157	1.225	1.016	Yes
PHASEGATE 4→0	1.218	1.257	1.271	1.251	1.027	No
PHASEGATE 4→1	1.573	1.177	1.170	1.172	1.042	Yes
PHASEGATE 4→2	1.789	1.289	1.384	1.352	1.054	No

The zero-decode-cap policies have higher TPOT than their one-cap counterparts in all three repetitions (Table A2); rounding does not explain the ordering. The cause remains unresolved, so calibration measures each candidate directly without assuming monotonicity or attributing the differences solely to noise. Fixed-1’s maximum ratio of 1.247 is close to the 1.25 threshold; its seven passing held-out runs support this configuration, not robustness to arbitrary background load.

A.3 M2 Output-Length Replication

Table 3 summarizes the device comparisons in the main text. The M2 Mini uses its separately calibrated baseline and the same fixed policies at both output lengths. The median PHASEGATE/Fixed throughput ratio decreases from $1.8\times$ across five pairs at 128 output tokens to $1.4\times$ across three pairs at 512 tokens.

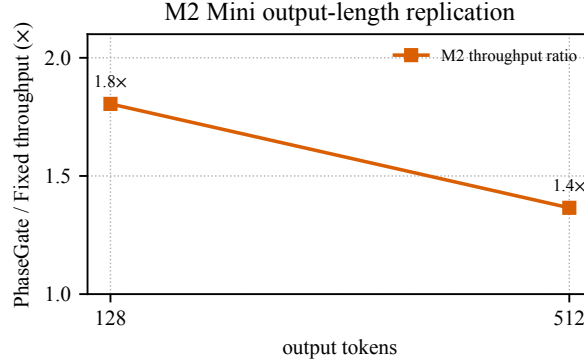


Figure A1: M2 Mini output-length replication with unchanged PHASEGATE 4→1 and FIXED-1 caps. The 128-token point summarizes five pairs and the 512-token point summarizes three; M2 is analyzed separately from M4.

A.4 Intermittent Retrieval Demand

At 5% and 25% duty, 2.0-s backlogged bursts are separated by mean idle gaps of 38 s and 6.0 s, respectively. Each duty, including continuous demand, uses five matched three-policy comparisons. The rounded 95% paired-bootstrap intervals for the full-wall-clock PHASEGATE/Fixed throughput ratio are [1.8, 2.0], [2.0, 2.1], and [2.0, 2.0], respectively. These experiments vary availability of queued work, not a distribution of individual request arrivals or deadlines.

A.5 CPU Trace Accounting

Section 4.3 reports the CPU service and drain measurements. The M4 calibration, mechanism, and scheduler evaluations use single-threaded FAISS calls with 16 vector queries per non-preemptive chunk, HNSW `efSearch=128`, and `top-k = 10`. A cap counts concurrent calls, and throughput counts completed vector queries. The feeder groups 4,096 queries per task; its task-level latency field is not individual-query response time.

Active calls are admitted calls that have not completed; we verify their count against every logged event. Chunk service time runs from its recorded start to completion and excludes queue waiting. Service-time statistics include chunks whose start and completion fall within the interval from the first request start to the last request completion. For each decode entry, the drain statistic in Table 4 includes any later increase above the cap within the same decode interval: the count must fall to at most one and remain there. Completion of all calls active at decode entry is a separate statistic, tracking only that initial set through its last completion; it is zero when that set is empty. For both statistics, we compute the p95 separately within each run, report the median across seven runs, and separately report the largest observation across all seven traces.

B Related Work

Phase-Structured LLM Serving. Server systems exploit the different resource profiles of pre-fill and decode inside the inference stack. Splitwise and DistServe assign the phases to separately provisioned GPU pools, Sarathi-Serve chunks prefills to avoid stalling ongoing decodes, and speculative decoding reduces sequential decode work [14, 6, 15, 16]. PHASEGATE is complementary: it leaves LLM execution unchanged on one on-device GPU and uses the phase boundary to regulate an independent CPU retrieval workload.

Shared-Memory Interference. BWLOCK++ activates CPU memory-traffic throttling around GPU execution and profiles bandwidth budgets to protect integrated-GPU kernels [8]. Sereno detects mobile memory contention and yields background LLM inference to foreground applications [17]. Adjacent GPU-sharing work such as REEF instead provides kernel preemption and controlled co-execution [18]. FusionML distinguishes prefill from decode, partitioning prefill matrix operations across the Apple-Silicon CPU and GPU while leaving decode on the GPU [9]. The control targets are CPU bandwidth budgets in BWLOCK++, LLM operator partitioning in FusionML, and admission of independent CPU retrieval in PHASEGATE. Our fixed-cap and TimeGate comparisons measure the throughput–latency effects of retrieval admission and phase alignment.

On-Device Retrieval. Apple’s Foundation Models guidance presents retrieval-augmented generation as a way to fetch relevant context for an on-device model [19]. MobileRAG reduces vector-search and context-processing costs on mobile devices, and MUSE coordinates interactive vector queries with background index maintenance on mobile SoCs [20, 4]. FAISS and HNSW provide established approximate-nearest-neighbor mechanisms for this retrieval kernel [10, 11]. These systems motivate the retrieval workload studied here.

Agentic CPU–GPU Scheduling. Recent studies characterize CPU-heavy tool execution, map heterogeneous tools across CPUs and GPUs, or co-schedule multi-turn LLM–tool workflows [3, 21, 22]. They optimize end-to-end heterogeneous workflows, primarily on server platforms. PHASEGATE compares fixed, phase-aligned, and time-matched phase-blind admission of pending CPU retrieval while measuring TTFT and TPOT on unified memory. Evaluating end-to-end agent dependencies and waiting times remains future work.